IT PROJECT MANAGEMENT · CHAPTER 07

Quality Management

Quality management ensures the project satisfies the needs for which it was undertaken, through planning, assurance and control activities integrated with the engineering work.

PMBOK Guide 7th ed. **ISO 9001 · ISO/IEC 25010 · CMMI** **Scrum Guide 2020**

AGENDA

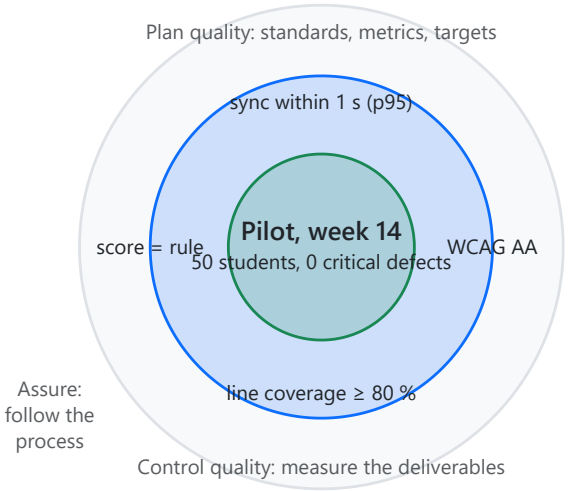
What we will cover

1. Quality, grade and the cost of quality (prevention, appraisal, failure)
2. Quality standards for IT: ISO 9001, ISO/IEC 25010 product quality, CMMI
3. Quality management plan and quality metrics for software projects
4. Quality assurance versus quality control
5. Seven basic quality tools: cause-and-effect, flowcharts, check sheets, Pareto, histograms, control charts, scatter diagrams
6. Reviews, inspections and pair programming as assurance practices
7. Testing levels and acceptance criteria as quality control (link to Software Engineering chapter 6)
8. Definition of done, coding standards and continuous integration
9. Continuous improvement: PDCA, retrospectives, root-cause analysis
10. Quality audits and process improvement plans
11. Chapter Quiz
12. Lab-07

Click any item to jump directly to that topic.

Why quality is a project management problem, not only a testing problem

- Chapters 04-06 defined **what** the Realtime Quiz project delivers, **when** and at **what cost**. Quality asks: will the result actually satisfy the needs it was undertaken for?
- The sponsor (Head of IT) will judge the pilot in week 14 on things no Gantt chart shows: does the leaderboard update within a second, does the score match the rule, does anything crash with 50 students connected?
- Quality is **planned** (standards, metrics, targets), **assured** (the process is followed) and **controlled** (deliverables are measured) throughout, not inspected in at the end.
- Software Engineering chapter 6 taught you *how* to test; this chapter teaches you how to decide *what quality means*, *how much* to invest and *how to prove* the target was reached.



Quality questions per chapter

Ch. 04 scope: which requirements?

Ch. 05 schedule: when are gates?

Ch. 06 cost: how much to spend?

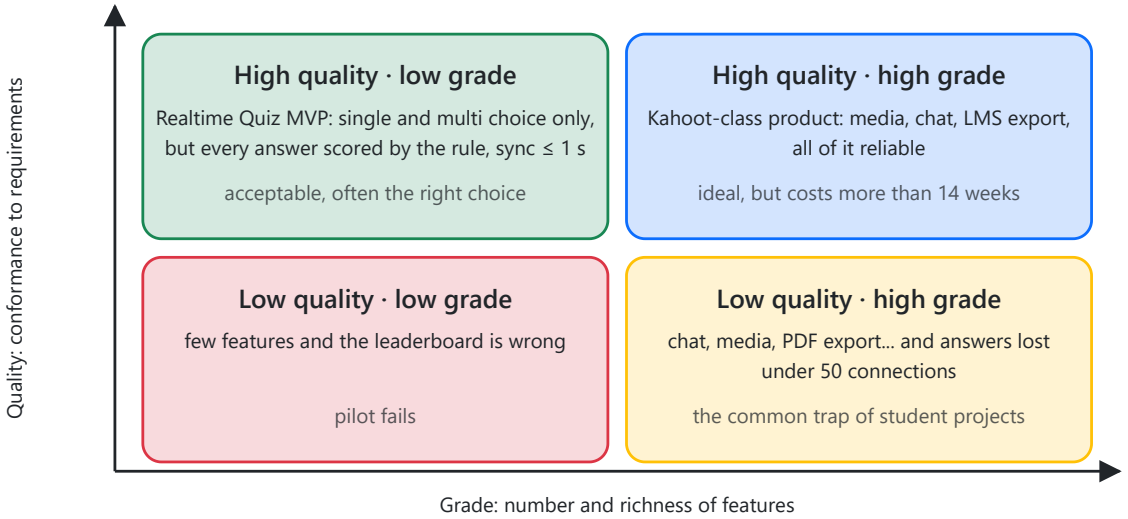
Ch. 07 quality: what is "good"?

SE Ch. 06: how to test it?

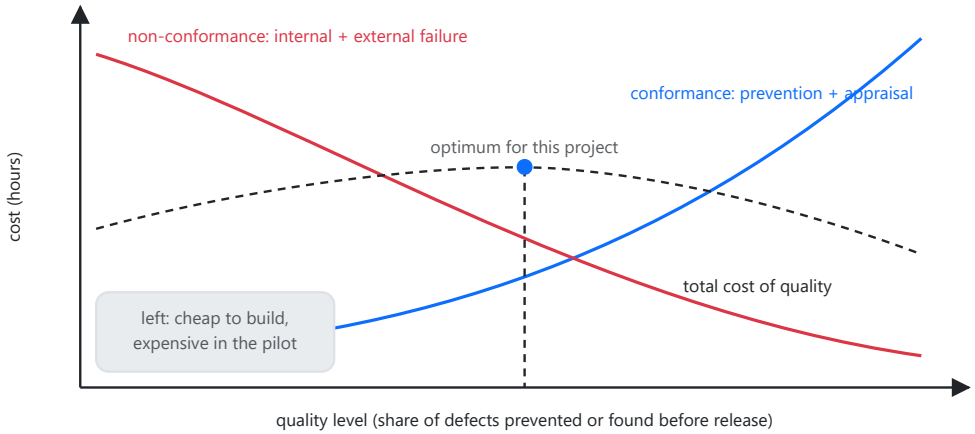
Quality is built in by the process, then confirmed by measurement.

Quality is not grade PMBOK Guide 7th ed.

- **Quality**: the degree to which a set of inherent characteristics fulfils requirements (ISO 9000 definition, used by the PMBOK Guide). Measured against what was promised.
- **Grade**: a category for deliverables with the same use but different features. A quiz with only single-choice questions is *low grade*; if it never loses an answer it is still *high quality*.
- Low grade can be a deliberate, acceptable decision (the Realtime Quiz "Non-Goals": no LMS integration, no free-text questions). Low quality is never acceptable.
- **Precision** (repeatable measurements) is not **accuracy** (close to the true value): a timer that is always 400 ms late is precise and wrong.
- Two modern principles: quality is **customer-defined** and **prevention beats inspection**.



Cost of quality: conformance versus non-conformance



Category	Realtime Quiz examples
Prevention	coding standard, definition of done, training on STOMP, pair programming on the scoring lock
Appraisal	code reviews, unit and integration tests, load test with 100 simulated students, sprint demo
Internal failure	rework after a failed review, fixing a red CI build, re-running a load test
External failure	hotfix during the pilot, lost session data, lecturer re-runs a quiz, reputation with the sponsor

Rule of ten: a defect found in the pilot costs about ten times a defect found in code review, and a hundred times one prevented by a clear requirement. That is why prevention hours are the best-spent quality hours.

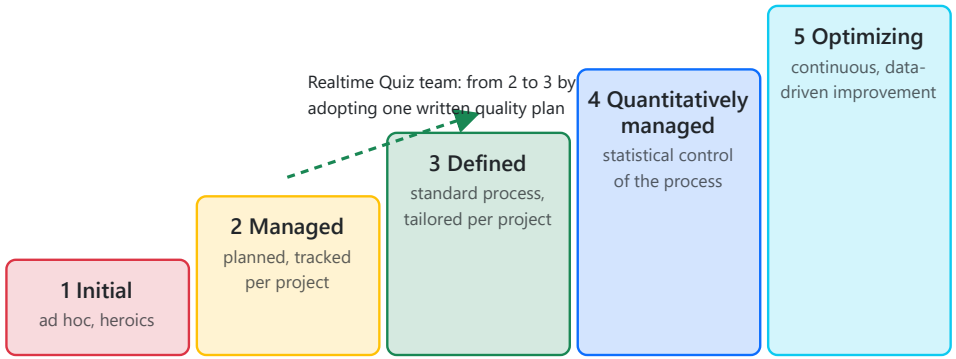
```
Worked example: cost of quality, Realtime Quiz (blended 12 USD/h)
Category            Hours   Cost      Share of CoQ
Prevention             40    480 USD   26.7 %
Appraisal              60    720 USD   40.0 %
Internal failure       30    360 USD   20.0 %
External failure       20    240 USD   13.3 %
Total cost of quality 150  1 800 USD  100 %
Conformance      = 40 + 60 = 100 h (66.7 %)
Non-conformance  = 30 + 20 =  50 h (33.3 %)
If the Lab-06 effort baseline is 600 h: CoQ = 150/600 = 25 %
```

Process standards: ISO 9001 and CMMI

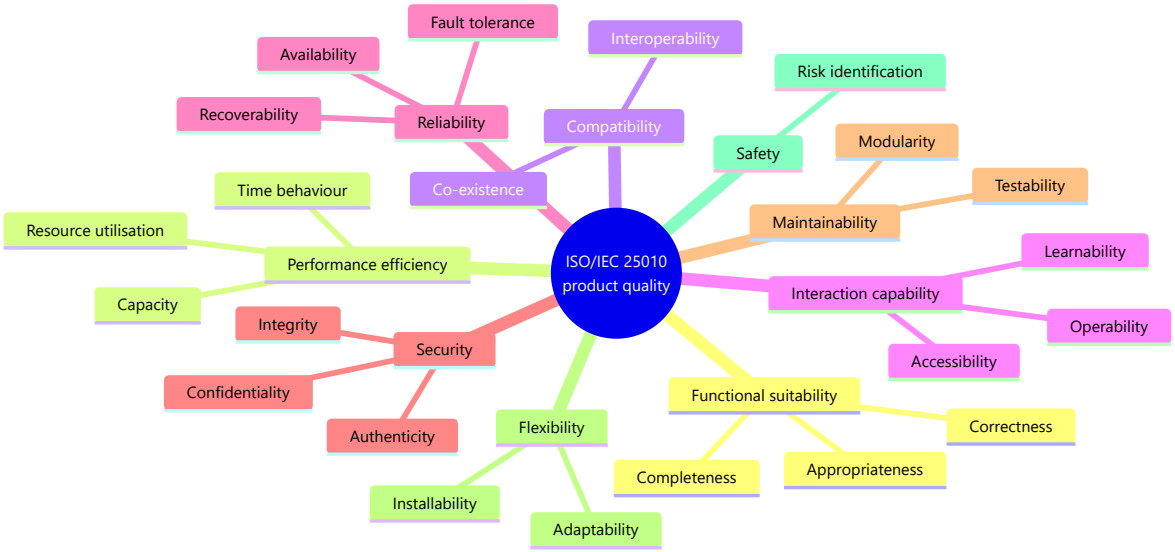
Standard	What it covers	Use on a student project
ISO 9001:2015	Requirements for a quality management system: customer focus, leadership, process approach, evidence-based decisions, improvement. Certifiable.	Borrow the principles: written process, records (review logs, test reports), corrective actions.
ISO/IEC 25010:2023	Product quality model: nine characteristics (next slide) with sub-characteristics.	Vocabulary for non-functional requirements and metrics.
CMMI v3.0	Capability Maturity Model Integration: five maturity levels describing how predictable and improving an organisation's processes are.	A ladder to judge your own team: are we at level 1 (heroics) or level 3 (a defined, repeated process)?
ISO/IEC/IEEE 29119	Software testing processes, documentation and techniques.	Names for test levels and the test plan (SE chapter 6).

⚠ A standard tells you *what* a good process contains, never *how much* of it your 14-week project needs. Tailor: write down which clauses you adopt and which you skip, with a reason.

CMMI Institute (ISACA), CMMI v3.0: maturity levels, described in our own words



ISO/IEC 25010: a vocabulary for product quality



Map every non-functional requirement of the Realtime Quiz plan to a characteristic; a characteristic with no requirement is a gap to discuss with the sponsor.

Characteristic	Realtime Quiz requirement
Functional correctness	FR-C1 score = points × (0.5 + 0.5 × remaining/limit)
Time behaviour	broadcast ≤ 500 ms on LAN, ack ≤ 300 ms, sync ≤ 1 s
Capacity	20 sessions × 100 participants on 2 vCPU
Recoverability	reconnect resumes score (FR-S9)
Accessibility	keyboard reachable options, WCAG AA contrast
Confidentiality	JWT, role checks, no secrets in Git
Testability	80 % line coverage on service and realtime
Compatibility	last two versions of four browsers, PostgreSQL and MySQL

The 2023 edition renamed *usability* to *interaction capability*, *portability* to *flexibility* and added *safety*.

The quality management plan: a subsidiary plan you can audit against

- Output of **Plan Quality Management** (PMBOK Guide): how the team will implement the quality policy and prove conformance.
- Short is fine. Four pages that the team actually reads beat forty that nobody opens.
- Every statement must be **checkable**: "reviews will be done" is not; "every merge request has one approval from a non-author within 24 h" is.
- Inputs: scope statement and requirements (Lab-04), schedule with gates (Lab-05), cost baseline (Lab-06), the organisation's standards.
- Companion outputs: **quality metrics** (next slide) and **checklists** (code review, release).

⚠ A quality plan without an owner per activity and a slot in the schedule is a wish list. Lab-07 Task 5 puts every activity on the Lab-05 calendar.

```
# Quality Management Plan · Realtime Quiz   (v1.0, sprint 1)

## 1. Quality objectives
Pilot in week 14 with 50 students: 0 open critical defects,
leaderboard sync p95 < 1 s, every score equals FR-C1.

## 2. Standards and references
ISO/IEC 25010 vocabulary; Google Java Style; WCAG 2.2 AA;
Definition of Done v1 (see section 6).

## 3. Roles
Quality owner: lecturer (PM). Reviewer rota: 4 students,
no one reviews their own merge request.

## 4. Metrics (target, method, frequency)     -> quality-metrics.csv
## 5. Assurance activities (process)
Code review on 100 % of merges · CI on every push ·
sprint demo every 2 weeks · quality audit in weeks 6 and 12
## 6. Control activities (product)
Unit/integration tests, load test (week 11), acceptance
test with 5 students (week 13), release checklist per tag
## 7. Defect management
GitHub issues, labels: category, severity (S1-S4), phase found
## 8. Cost of quality budget                  -> 150 h, see Lab-06
```

Quality metrics: a target is only real when you know how to measure it

Metric	Target	Measurement method	When
Leaderboard sync latency, p95	< 1 000 ms	client logs (server timestamp in LEADERBOARD event − receive time); 95th percentile over a session	every load test, pilot
Answer ack latency, p95	< 300 ms	same method on ANSWER → ANSWER_ACK	every load test
Line coverage, `service` + `realtime`	≥ 80 %	JaCoCo report in `mvn verify`, CI fails below	every push
Defect density (escaped)	≤ 0.5 / KLOC	defects found in pilot ÷ KLOC of Java main code (8 KLOC → ≤ 4)	after pilot
Crash-free sessions	≥ 99 %	sessions reaching ENDED ÷ all started sessions, from the database	weekly, pilot
Timer drift	< 200 ms	client deadline vs server deadline after offset correction, 2 devices	sprint demo
Review coverage	100 %	merged MRs with ≥ 1 non-author approval ÷ merged MRs	weekly
Build health	≥ 90 % green on `main`	GitHub Actions run history	weekly

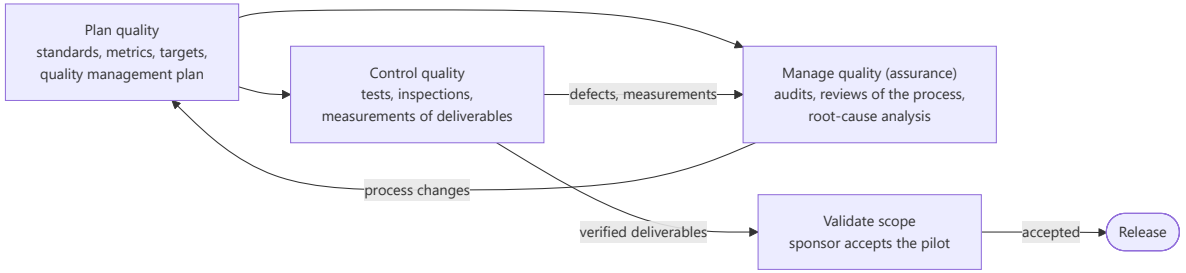
```
Anatomy of a metric (fill all six or delete the metric)
Name ........ Leaderboard sync latency (p95)
Target ...... < 1000 ms   (tolerance: 1200 ms for 1 sprint)
Method ...... t_receive - t_server from client log
Sample ...... all LEADERBOARD events of one session
Frequency ... each load test, and the pilot
Owner ....... realtime developer (student C)

Why p95, not the average?
100 events: 95 at 300 ms, 5 at 4 000 ms
average = (95*300 + 5*4000)/100 = 485 ms  (looks fine)
p95     = 300 ms, p99 = 4 000 ms (5 students waited 4 s)
Report the percentile the users feel.
```

Tolerance is the range a result may vary and still be accepted; a **control limit** (slide 15) is a statistical boundary of the process. Do not confuse them.

Assurance looks at the process, control looks at the product

	Quality assurance (Manage Quality)	Quality control
Question	Are we working the right way?	Is this deliverable right?
Object	the process, the plan, the standards	a merge request, a build, a release
Timing	during execution, proactive	after a deliverable exists, reactive
Tools	audits, process analysis, checklists, root-cause analysis, retrospectives	testing, inspection, measurement, control charts
Realtime Quiz	"was every MR reviewed with the checklist?" · "is the DoD applied?"	"does the 200-thread test pass?" · "p95 latency < 1 s?"
Output	improved process, audit report, change requests	verified deliverables, defect list, measurements

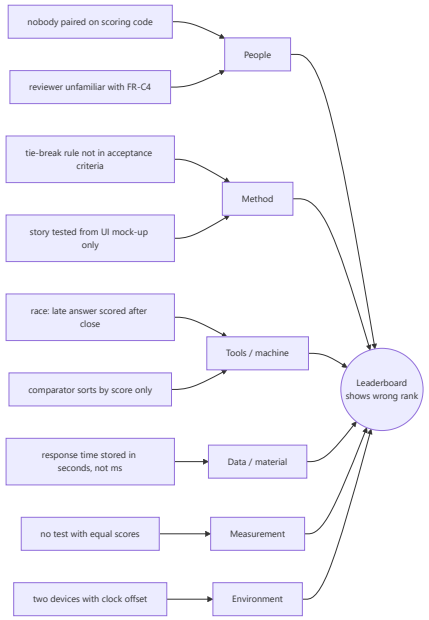


PMBOK Guide 6th ed. process names: Plan Quality Management, Manage Quality, Control Quality. Verified deliverables then go to *Validate Scope* for customer acceptance (chapter 04).

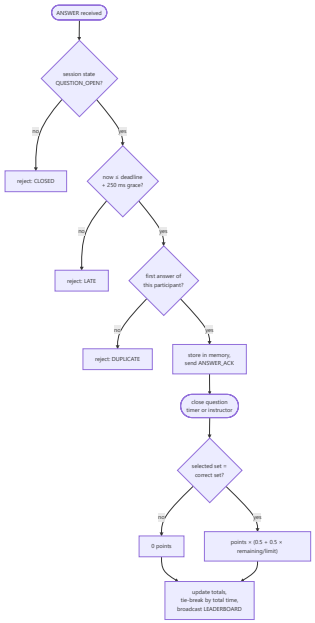
The seven basic tools, and tool 1: the cause-and-effect diagram

Tool	Question it answers	Slide
Cause-and-effect (Ishikawa, fishbone)	What could cause this problem?	11
Flowchart	Where in the process can it go wrong?	12
Check sheet	How often, where, when does it occur?	12
Pareto chart	Which few causes make most of the trouble?	13
Histogram	How is a measurement distributed?	14
Control chart	Is the process stable or drifting?	15
Scatter diagram	Are two variables related?	16

Order of use in a real investigation: check sheet → Pareto → fishbone on the top category → flowchart to locate it → control chart to confirm the fix held.



Tools 2 and 3: flowchart of the answer-scoring process, check sheet of failures

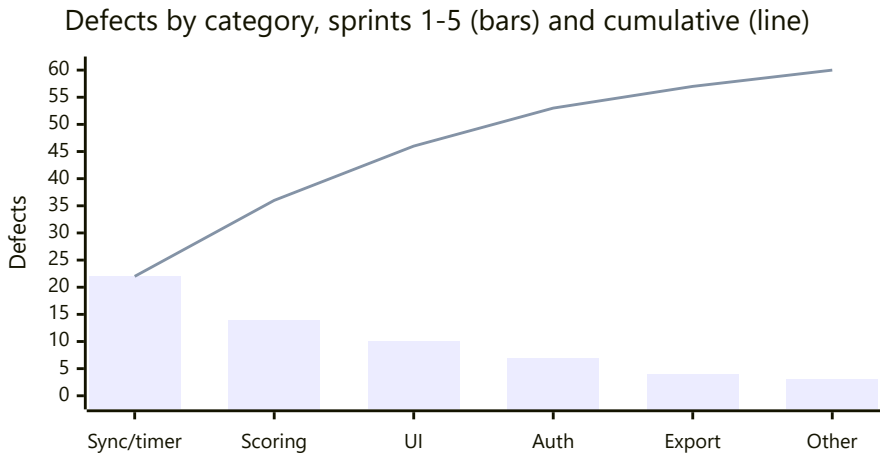


The flowchart shows *where* a defect can enter (each decision diamond is a test case). The **check sheet** is the simplest tool: a tally kept while observing, here during two weeks of sprint testing.

Failure observed	Wk 7	Wk 8	Total
Late answer accepted after close	||||	||	6
Duplicate answer counted twice	|		1
Wrong points for fast correct answer	|||	||||	7
Tie ordered by join time, not total time	|||||	|||	8
Leaderboard delayed > 1 s	||	|||	5
Reconnect loses score	|	|	2
Total	16	13	**29**

```
Check sheet rules
1. Decide the categories BEFORE observing (from the flowchart).
2. One tally per occurrence, never from memory afterwards.
3. Record where and when; the sheet feeds the Pareto chart.
```

Tool 4: Pareto chart of 60 categorised defects



The line is the cumulative count; read it against the right-hand total of 60 (100 %). Mermaid draws one y-axis, so the cumulative percentages are in the table.

```
Worked example: Pareto analysis of the defect log (60 defects)
Step 1 count per category, Step 2 sort descending,
Step 3 cumulative count and cumulative %.

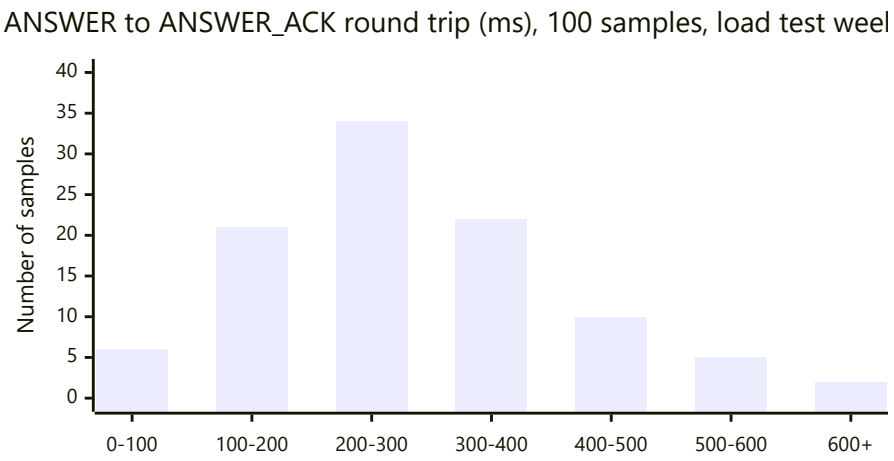
Category      Count   %      Cumulative   Cum. %
Sync/timer      22   36.7 %      22        36.7 %
Scoring         14   23.3 %      36        60.0 %
UI/layout       10   16.7 %      46        76.7 %
Auth             7   11.7 %      53        88.3 %
Export           4    6.7 %      57        95.0 %
Other            3    5.0 %      60       100.0 %

Reading: 2 of 6 categories (33 %) cause 60 % of defects.
Decision: fishbone + root-cause analysis on Sync/timer first,
          then Scoring. Do not start with Export.

Spreadsheet: sort by B desc; C2 = B2/SUM($B$2:$B$7)
             D2 = B2 ; D3 = D2+B3 ; E2 = D2/SUM($B$2:$B$7)
```

The 80/20 shape is a heuristic, not a law: if the bars are almost flat, categorise differently (by component, by phase found, by severity-weighted count) until a few categories stand out.

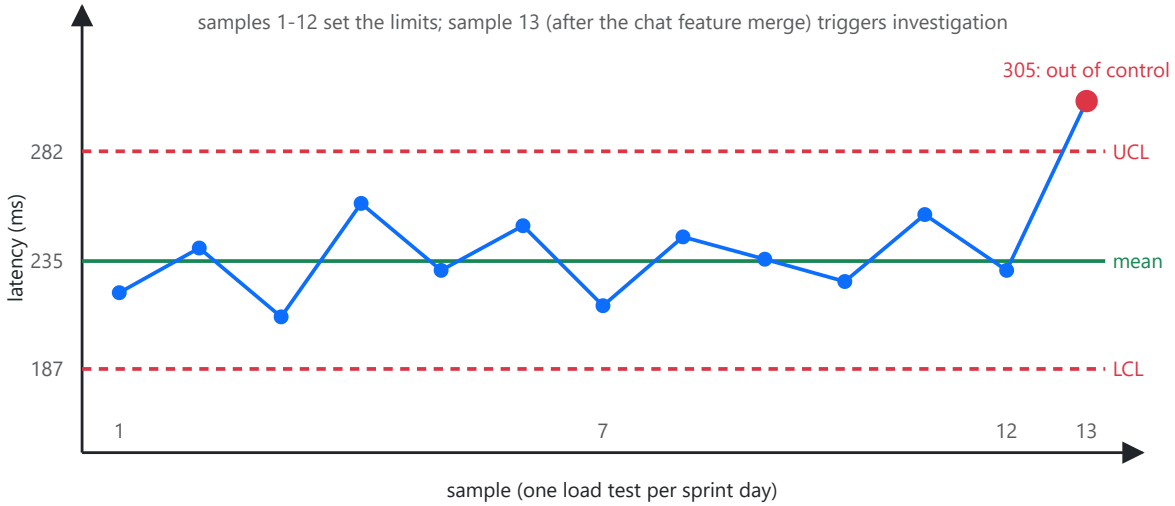
Tool 5: histogram of answer round-trip times



- A histogram shows the **shape** of one variable: centre, spread, skew, outliers. A Pareto chart ranks categories; a histogram bins a continuous measurement.
- Bins: equal width, 5 to 10 bins for 50 to 200 samples. Here 100 ms bins.
- Reading: centre around 250 ms, right-skewed tail. Target "ack < 300 ms" is met by 61 of 100 samples; the p95 lies in the 500-600 bin, above target.
- The tail (600+, 2 samples) is what users notice. A bimodal shape (two humps) usually means two populations mixed: LAN and Wi-Fi clients, or two server states.

```
Bin (ms)   Count   Cum.   Cum. %      Spreadsheet
0-100          6      6      6 %      =COUNTIFS(A:A,">=0",A:A,"<100")
100-200       21     27     27 %      =FREQUENCY(A2:A101,{100;200;...})
200-300       34     61     61 %  <- target met up to here
300-400       22     83     83 %
400-500       10     93     93 %
500-600        5     98     98 %  <- p95 is in this bin
600+           2    100    100 %
```

Tool 6: control chart of broadcast latency with UCL and LCL



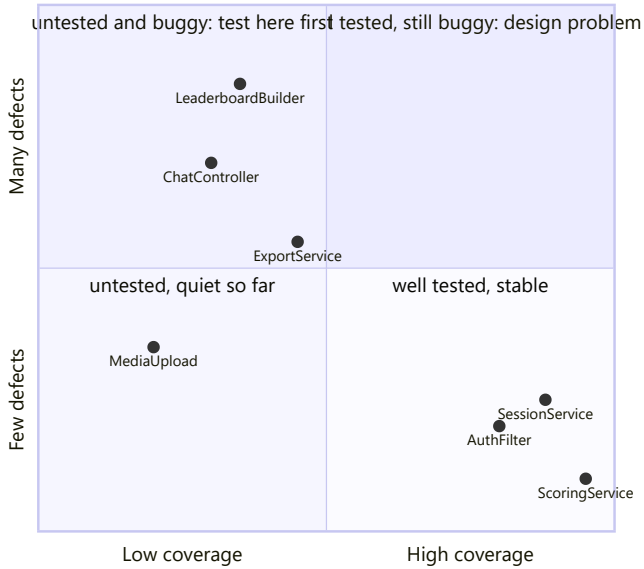

```
Worked example: 12 samples of LEADERBOARD latency (ms)
220 240 210 260 230 250 215 245 235 225 255 230
mean  x̄ = 2815 / 12 = 234.6 ms
s = sqrt( Σ(x - x̄)² / (n-1) ) = 15.9 ms
UCL = x̄ + 3s = 234.6 + 47.7 = 282.2 ms
LCL = x̄ - 3s = 234.6 - 47.7 = 187.0 ms
Spreadsheet: =AVERAGE(B2:B13)  =STDEV.S(B2:B13)
             =B15+3*B16        =B15-3*B16

Sample 13 = 305 ms > UCL  -> special cause, investigate
Rule of seven: 7 points in a row on one side of the
mean = the process drifted, even inside the limits.
```

⚠ Control limits describe the *process*; the specification limit (1 000 ms) describes the *customer's need*. A process can be in control and still miss the specification.

Tool 7: scatter diagram of test coverage against defects found

Line coverage per component vs defects found in sprints 1-5



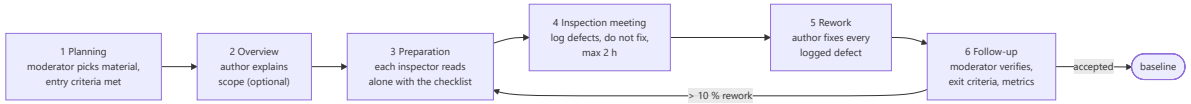
- A scatter diagram plots two variables per item to look for a **relationship**: here a negative one, more coverage goes with fewer defects.
- Correlation is not cause: `LeaderboardBuilder` may be buggy because it is complex, and complex code is also harder to test.
- Decision from the picture: raise coverage on `LeaderboardBuilder` and `ChatController` before the pilot; leave `MediaUpload` for later.

```
Component            Coverage   Defects
ScoringService          95 %        1
SessionService          88 %        3
AuthFilter              80 %        2
ExportService           45 %        6
LeaderboardBuilder      35 %       10
ChatController          30 %        8
MediaUpload             20 %        4
Spreadsheet: =CORREL(B2:B8, C2:C8)  -> about -0.85
(strong negative correlation; -1 perfect, 0 none)
```

Reviews find defects where they are cheapest: before the code runs

Practice	Formality	Realtime Quiz use
Pair programming	continuous, informal	the per-session lock in `SessionService`: two heads on the concurrency code
Peer review (merge request)	light, asynchronous, checklist	every merge to `main`, one non-author approval within 24 h
Walkthrough	author presents, team asks	sprint demo of the host console to the lecturer
Technical review	planned meeting, minutes	architecture review of the WebSocket design before phase 3
Inspection (Fagan)	formal roles, entry and exit criteria, metrics	scoring rules FR-C1 to FR-C4 before the MVP checkpoint

💡 Review speed matters: above about 400 lines per hour reviewers stop finding defects. Keep merge requests under 400 changed lines.



Roles in an inspection: moderator, author, reader, recorder, inspectors. Metrics recorded: defects per page or per 100 lines, preparation rate, meeting rate. These feed the Pareto chart and the cost-of-quality appraisal hours.

Worked example: the code review checklist of the Realtime Quiz team

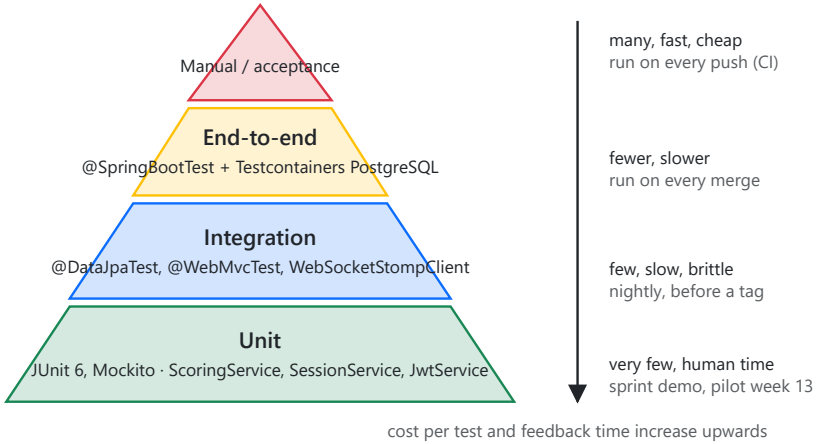
```
# Code review checklist · Realtime Quiz   (reviewer ticks each line)

## Scope and traceability
- [ ] MR title names the story and its FR ids (e.g. FR-C4)
- [ ] Change is < 400 lines; otherwise split before review
## Correctness
- [ ] Every FR in the story has at least one test that fails
      without the change (reviewer checked one by reverting)
- [ ] Session state and deadline checked inside the lock
- [ ] Time values are in one unit (ms) end to end
## Tests and CI
- [ ] CI green; JaCoCo on service+realtime not below 80 %
- [ ] No test uses Thread.sleep to "wait for" the broker
## Security
- [ ] Endpoint has a role check; no secret in code or YAML
- [ ] Input validated (Jakarta Validation) with problem detail
## Maintainability
- [ ] Names follow Google Java Style; no commented-out code
- [ ] New STOMP event added to the WebSocket Design table
## Result: [ ] approve   [ ] approve with nits   [ ] request changes
Defects found: ___  (logged with category + severity in issues)
```

- A checklist turns "look at the code" into a repeatable **assurance** activity: the same questions every time, so the audit in week 6 can verify it was used.
- Keep it to one screen; retire an item that has not caught a defect in three sprints, add an item for every escaped defect category (Pareto feedback).
- Reviewers log **defects found** per review: the number that feeds appraisal cost and the phase-found column of the defect log.
- **Pair programming** is prevention, not appraisal: the defect never gets written. Use it for the riskiest 10 % of the code (concurrency, scoring, security), not for everything.

⚠ "Looks good to me" without a checklist tick is review theatre. The rubric of Lab-07 deducts points for checklist items a reviewer cannot answer yes or no to.

Test levels: the pyramid from Software Engineering chapter 6, seen by the project manager



Level (SE ch. 6)	Realtime Quiz control activity	Gate
Unit	formulas, tie-breaks, state transitions, join-code generator	CI green on every push
Unit (concurrency)	200 concurrent `submitAnswer` calls: exactly one answer per participant	must pass before MVP checkpoint (week 7)
Repository / web slice	derived queries, Flyway on H2, 401/403 paths, CSV content type	every merge
WebSocket	connect with and without JWT, private RESULT to the right user	every merge
End-to-end	register two users, run a 3-question session, download CSV	before each tag
Load	100 simulated students, p95 latency, no lost answer	week 11 hardening
Acceptance	5 students, two devices, reconnect during a question	week 13, sponsor sign-off

💡 The PM's job is not to write the tests but to put each level's **gate** on the schedule and to refuse a release that skipped one.

Acceptance criteria turn a requirement into a pass/fail check

```
Story S-14: "As a student I earn more points for a faster answer"
Traces to: FR-C1, FR-C2, FR-C4

AC-1  Given a 1000-point question with a 20 s limit
      When a correct answer arrives with 20 s remaining
      Then the participant earns 1000 points
AC-2  Given the same question
      When a correct answer arrives with 10 s remaining
      Then the participant earns 1000 * (0.5 + 0.5 * 10/20) = 750
AC-3  Given the same question
      When a correct answer arrives with 0.4 s remaining
      Then the participant earns round(1000 * 0.51) = 510
AC-4  When the answer is wrong or missing   Then 0 points (FR-C2)
AC-5  Given two participants with equal totals
      When the leaderboard is built
      Then the lower total response time ranks first (FR-C4)
AC-6  When an answer arrives 300 ms after the deadline
      Then it is rejected as LATE (grace is 250 ms)

Each AC becomes one JUnit test named after it: ac2_halfTime_750pts
```

- **Acceptance criteria** are the quality control of a story: written before the code, testable, owned by the product owner (lecturer) and agreed with the team.
- Given / When / Then keeps them concrete. Numbers, not adjectives: "fast" is not testable, "750 points at half time" is.
- Cover the boundaries: full time, half time, last moment, wrong, late, tie. Most escaped Realtime Quiz defects live at a boundary (the 250 ms grace, equal scores).
- **Test gate**: a scheduled point where a defined set of tests must pass before work continues (merge, tag, MVP checkpoint, pilot). Gates appear as milestones in the Lab-05 schedule.

Gate	Must pass	Who signs
Merge	unit + slice tests, coverage ≥ 80 %, review approved	reviewer
Sprint demo	all AC of the sprint's stories, demo on phone viewport	lecturer
MVP checkpoint (wk 7)	every Must feature end to end, concurrency test	lecturer + sponsor
Pilot go/no-go (wk 13)	load test p95 < 1 s, 0 open S1/S2 defects, release checklist	sponsor

Definition of done: the team's shared quality bar Scrum Guide 2020

```
# Definition of Done · Realtime Quiz   (applies to every story)

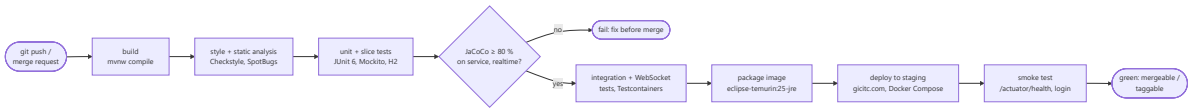
A story is Done only when ALL of the following are true:
1. Every FR id listed in the story has an automated test
   named after its acceptance criterion, and all pass.
2. `./mvnw verify` is green in CI; JaCoCo line coverage on
   `service` and `realtime` is at least 80 %.
3. The merge request was reviewed with the checklist by a
   non-author and approved; all logged defects are fixed.
4. New REST endpoints appear in `/v3/api-docs`; new STOMP
   events are listed in the WebSocket Design table.
5. Flyway migrations apply on PostgreSQL, MySQL and H2.
6. The feature was demonstrated on a desktop and a phone-sized
   viewport during the sprint demo.
7. No open S1 (critical) or S2 (major) defect on the story.
8. The plan or README is updated if the design changed.

Changing this list requires a team decision in a retrospective.
```

- The Scrum Guide 2020 defines the **Definition of Done** as a formal description of the state of the Increment when it meets the quality measures required for the product. If a story does not meet it, it does not count as done and is not shown at the review.
- It is **prevention**: the same checklist for every story stops half-finished work from accumulating as hidden rework (internal failure cost).
- Compare with acceptance criteria: AC are *per story* (what this feature does), the DoD is *for all stories* (how finished work looks).
- **Coding standards** remove debate from reviews: adopt an existing one (Google Java Style), enforce it with a formatter and Checkstyle in the build so reviewers spend time on logic, not braces.

Standard element	Enforced by
formatting, naming, imports	`spotless:check` or Checkstyle in `verify`
no `System.out`, no empty catch	static analysis (SpotBugs, SonarLint)
public API documented	review checklist item
commit message format	Git hook or CI lint

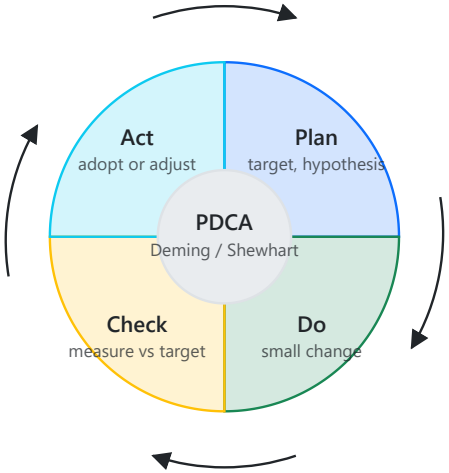
Continuous integration automates the appraisal and makes the gates cheap



- **Continuous integration**: every change is merged to a shared branch at least daily and verified by an automated build with tests. Feedback in minutes instead of at the sprint end.
- Each stage is a **quality gate**; a red stage blocks the merge. Gates encode the DoD so nobody has to remember it.
- For the project manager, CI produces **metrics for free**: build health, coverage trend, test duration, failure categories for the Pareto chart.
- Keep the pipeline under 10 minutes; move slow end-to-end tests to a nightly run so developers do not start skipping CI.

```yaml
# .github/workflows/ci.yml (excerpt)
on: [push, pull_request]
jobs:
  verify:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-java@v4
        with: { distribution: temurin, java-version: 25 }
      - run: ./mvnw -B verify      # tests + JaCoCo gate
```

Plan, Do, Check, Act: the improvement loop behind every retrospective



One turn of the wheel, sprint 4

Plan: p95 leaderboard latency 1 350 ms; target < 1 000 ms. Hypothesis: 10 top rows are serialised per participant.

Do: build the leaderboard once, broadcast it, send only the own rank privately.

Check: load test 100 clients: p95 = 640 ms.

Act: adopt; add the design rule to the review checklist; new target 500 ms for sprint 6.

- **Plan**: pick one problem from the data (Pareto, control chart), state a measurable target and a hypothesis about the cause.
- **Do**: try the change on a small scale, one sprint, one component.
- **Check**: measure with the same method as before. No measurement, no check.
- **Act**: if it worked, make it the standard (DoD, checklist, coding rule); if not, learn and turn the wheel again.
- Kaizen: many small turns beat one big "quality initiative". Each Scrum retrospective is one turn (Scrum Guide 2020: the team identifies the most helpful changes and addresses them as soon as possible).

```
Process improvement item (one row of the plan)
Problem ..... p95 leaderboard latency 1 350 ms (target 1 000)
Root cause .. per-participant serialisation of top-10 rows
Change ...... build once, broadcast, private own-rank only
Owner/when .. student C, sprint 5
Measure ..... load test, 100 clients, same script as sprint 4
Target ...... p95 < 1 000 ms; stretch 500 ms
Result ...... 640 ms (sprint 5) -> adopted, checklist updated
```

Root-cause analysis with 5 whys, and retrospective formats

```
5 whys · "Leaderboard shows wrong rank" (found in sprint demo, S2)
Symptom: two students with 2 350 points each; the slower one
         is shown first.
Why 1? The comparator orders by total score only; equal scores
       keep insertion order (join time).
Why 2? The tie-break FR-C4 (lower total response time first)
       was never implemented.
Why 3? FR-C4 was not in the acceptance criteria of story S-14;
       the AC were written from the UI mock-up.
Why 4? Stories are not checked against the FR list when the AC
       are written.
Why 5? The Definition of Done has no item "every FR id of the
       story has a test", so nothing forces the trace.
Root cause: a process gap (traceability), not a careless student.
Corrective: fix comparator + add test ac5_tie_lowerTimeFirst.
Preventive: DoD item 1 (trace + test per FR), checklist item
            "MR names FR ids". Verify: no tie defect in 3 sprints.
```

Stop asking "why" when the answer is a **process or system** you can change. If the answer is a person's name, you stopped too early.

Retrospective format	Prompts	Best when
Start / Stop / Continue	what to begin, end, keep	first retros, 15 min, small team
4 Ls	liked, learned, lacked, longed for	learning-heavy sprints (new tech: STOMP)
Mad / Sad / Glad	feelings first, then facts	after a tense sprint or a failed demo
Sailboat	wind (helps), anchor (slows), rocks (risks), island (goal)	mid-project, linking to risks (chapter 09)
Timeline	events on a line, mark highs and lows	long sprints, incident review
Data-driven	start from metrics: velocity, defects, CI health	every third retro, feeds the PDCA plan

```
# Retrospective · Sprint 4 (2026-11-16)   format: 4 Ls
Attendees: 4 students, lecturer          Facilitator: student B
Data: velocity 21/24 pts, 9 defects found (5 sync), CI 84 % green
Liked: pairing on the lock removed all race defects
Lacked: no test for ties; two red builds left overnight
Actions (max 3, owner, due, measure):
1. Add DoD item "test per FR id"  - lecturer, sprint 5, audit wk 12
2. Fix red main within 2 h        - all, sprint 5, CI ≥ 90 % green
3. Load test script in repo       - student C, sprint 5, p95 known
```

A quality audit checks that the plan is followed; the improvement plan closes the gaps

- A **quality audit** is a structured, independent review of whether project activities comply with the organisation's and the project's policies, processes and procedures (PMBOK Guide). It audits *the process*, so it is an assurance tool.
- Independent: not the person who did the work. On the Realtime Quiz project the lecturer audits, or students audit another team.
- Outputs: findings (conformance, non-conformance, observation), good practices to share, and **change requests** or improvement items with owners and dates.
- Two audits are enough for 14 weeks: week 6 (are the quality plan and DoD in use?) and week 12 (is the release checklist complete before the pilot?).

```
Quality audit · week 6 · auditor: lecturer · auditee: team
Ref  Check (evidence)                          Result
A1   Quality plan v1 in repo, reviewed by team  conform
A2   Last 10 MRs: non-author approval + ticked  8/10  NON-CONF
     checklist (evidence: MR pages)
A3   DoD applied: sprint 3 stories all have     conform
     tests per FR id (evidence: test names)
A4   Defect log has category + severity +       6/9   NON-CONF
     phase found for every defect
A5   CI history: red main fixed within 2 h      observation
Findings: 2 non-conformances, 1 observation
Actions -> process improvement plan (right)
```

```
# Process improvement plan · after audit week 6

## Item 1 (finding A2: 2 of 10 MRs merged without approval)
- Change:  branch protection on main, 1 approval required
- Owner / due: student A, week 7
- Measure: merged MRs with approval / all merged MRs (weekly)
- Target:  100 % from week 9; re-audit week 12

## Item 2 (finding A4: 3 of 9 defects logged without category)
- Change:  issue template with required category, severity, phase
- Owner / due: student D, week 7
- Measure: complete defect records / all defects
- Target:  100 % by week 8

## Item 3 (observation A5: red main left overnight)
- Change:  CI failure notification to the team chat
- Owner / due: student B, week 8
- Measure: hours main is red per week
- Target:  < 2 h per week on average
```

Finding type	Meaning	Follow-up
Conformance	evidence shows the process was followed	none, record it
Non-conformance	the process was not followed or does not exist	corrective action with owner and date, re-audit
Observation	works, but risk or waste is visible	improvement item, optional

⚠ An audit finding without a measurable follow-up target is a complaint. Every row of the improvement plan needs the Measure and Target columns filled, and a re-check date.

WRAP-UP

Best practices and common mistakes in quality management

1 Define quality with the customer

Every metric in the plan traces to a requirement the sponsor cares about (sync ≤ 1 s, correct scores). Coverage is a means, not a goal.

2 Spend on prevention first

DoD, acceptance criteria, coding standard, pairing on risky code. Each prevented defect saves ten times its cost in the pilot.

3 Put gates on the calendar

Reviews, audits, load test and acceptance test are schedule items with owners (Lab-05), and their hours are in the budget (Lab-06).

4 Measure percentiles and trends

p95 instead of averages, control charts instead of single readings, Pareto before choosing what to fix.

5 Ask why until you reach a process

Root causes are gaps in the DoD, the checklist or the plan. Blaming a person fixes nothing and hides the next defect.

6 Close the loop

Every retrospective action has an owner, a date and a measure; the next retro starts by checking the last one's results (PDCA).

✕ Mistake: testing as the quality plan

"We will test at the end" is control without assurance or prevention: defects are found when they are most expensive.

✕ Mistake: gold plating

Adding features nobody asked for raises grade, adds defects and eats the quality budget. Quality is conformance, not extras.

✕ Mistake: metrics without a method

"High performance", "good usability", "sufficient tests" cannot fail an audit because they cannot be measured. Delete or quantify.

Chapter quiz 10 questions

1. The Realtime Quiz MVP supports only single- and multi-choice questions, but every answer is scored correctly and the leaderboard syncs within 1 s. In PMBOK terms this deliverable is:

- Low quality, low grade
- High quality, low grade
- Low quality, high grade
- High quality, high grade

2. Prevention 40 h, appraisal 60 h, internal failure 30 h, external failure 20 h, all at 12 USD/h. What is the cost of non-conformance?

- 1 200 USD
- 600 USD
- 1 800 USD
- 240 USD

3. Which activity is quality assurance rather than quality control?

- Running the 200-thread concurrency test
- Measuring p95 leaderboard latency in a load test
- Auditing whether the last 10 merge requests used the review checklist
- Executing the acceptance test with five students

4. In the Pareto analysis of 60 defects (Sync/timer 22, Scoring 14, UI 10, Auth 7, Export 4, Other 3), what cumulative percentage do the top two categories reach?

- 36.7 %
- 76.7 %
- 60.0 %
- 50.0 %

5. Twelve latency samples have mean 234.6 ms and sample standard deviation 15.9 ms. The upper control limit (3 sigma) is about:

- 250 ms
- 266 ms
- 282 ms
- 1 000 ms

6. In ISO/IEC 25010, the requirement "a broadcast reaches all clients within 500 ms" belongs to which characteristic?

- Reliability
- Performance efficiency (time behaviour)
- Functional suitability
- Interaction capability

7. The team wants to brainstorm all possible reasons why the leaderboard sometimes shows the wrong rank. Which basic quality tool fits?

- Control chart
- Histogram
- Scatter diagram
- Cause-and-effect (fishbone) diagram

8. In the PDCA cycle, which step compares the load-test result with the target set earlier?

- Plan
- Do
- Check
- Act

9. According to the Scrum Guide 2020, the Definition of Done is:

- The acceptance criteria of one user story
- A formal description of the state of the Increment when it meets the quality measures required for the product
- The list of tests the QA engineer runs before release
- The sponsor's sign-off document for the pilot

10. A 5-whys analysis of the wrong-rank defect should stop when the answer is:

- The name of the student who wrote the comparator
- A process gap such as "the Definition of Done does not require a test per FR id"
- The first technical cause (the comparator sorts by score only)
- Five answers, whatever they are

WRAP-UP

Summary

Quality is conformance to requirements; grade is the feature level. Low grade can be fine, low quality never is.	A quality plan is auditable only when every metric has a target, a measurement method, a frequency and an owner.	Reviews with a checklist, pair programming on risky code and the test pyramid put appraisal where defects are cheap.
Cost of quality = prevention + appraisal (conformance) + internal + external failure (non-conformance); prevention is the cheapest hour.	Assurance checks the process (audits, reviews, RCA); control checks the product (tests, inspections, measurements).	Definition of done, coding standards and CI gates make the quality bar automatic and the same for every story.
ISO 9001 and CMMI describe the process, ISO/IEC 25010 gives the vocabulary for product quality and non-functional requirements.	Seven tools: check sheet, Pareto, fishbone, flowchart, histogram, control chart (mean ± 3s), scatter diagram.	Improve continuously: PDCA, retrospectives with measurable actions, 5 whys to a process cause, audits that end in an improvement plan.

Open Lab-07: 5 tasks + 1 challenge

Next chapter: 08 · Human Resource Management